Week 2 Reflection

Daniyal Shahid Iqbal

February 5, 2026

Artificial Neural Networks (ANNs) are computational models inspired by the biological brain. The fundamental building block of Deep Learning (DL) is the **Perceptron**. The perceptron computes a linear combination of inputs and weights, which is then passed through a non-linear activation function.

Depending on the task, the output can be a probability (for classification) or a continuous number (for regression).

Since our main focus is approximating complex functions (decision boundaries), we utilize non-linear activation functions. Common functions include:

- **Sigmoid:** Squashes output between 0 and 1; used for probabilities.
- **Tanh:** Squashes the neuron's output between -1 and 1; strictly increasing.
- **ReLU (Rectified Linear Unit):** Acts as a linear function for non-negative inputs and outputs zero for negative values.

These perceptrons are stacked to form deep networks consisting of an input layer, hidden layers (where feature extraction occurs), and an output layer.

Technical Note: When discussing the depth of a model, we include the output layer. For example, a "2-layer NN" consists of one hidden layer and the final output layer.

To train these networks, we use a loss function specific to the task:

- **Binary Classification:** Typically uses **Binary Cross Entropy** (the negative log-likelihood of a Bernoulli distribution), which penalizes wrong predictions.
- **Multinomial Classification:** Use **Cross Entropy with Softmax** to ensure probabilities sum to one.
- **Regression:** Generally relies on **Mean Squared Error (MSE)**.

The loss function optimization involves minimizing the loss function by adjusting the weight matrices via a process called **Gradient Descent**. Note that each layer has its own set of weights, and the loss is a function of the entire network's weights.

Gradient Descent Algorithm

1. Initialize weights
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $W^{t+1} \leftarrow W^t - \eta \frac{\partial J(W)}{\partial W^\top}$
5. Return weights

We also looked at the following questions and there answers:

1. **How to set initial weights?**

Weights are initialized randomly to break symmetry and ensure the network does not get stuck in a local minimum. There are many different ways to randomly initialize weights, but the most common method uses a Gaussian distribution with a mean of zero.

2. **How to compute gradient?**
   To determine the optimal weights, we must calculate the gradient. This process involves a **forward pass** to compute the loss, followed by **backpropagation** to calculate gradients via the chain rule. This is repeated for every weight in the network using gradients derived from later layers.

3. **How to choose the stepsize?**
   Weights are updated in the opposite direction of the gradient by a specific step size (learning rate). An appropriate step size helps avoid getting stuck in a local minimum (since the loss function is non-convex and has multiple minima).

   Note on Optimization Speed: Calculating gradients over an entire dataset is computationally expensive. To speed this up, we often use **Stochastic Gradient Descent (SGD)** or **Mini-Batch Gradient Descent**. Additionally, we use adaptive learning rate algorithms such as **Adagrad**, **RMSprop**, or **Adam** to manage learning rates effectively.

4. **When to stop?**
   We typically run the algorithm until convergence. However, over-optimization can cause the model to overfit the training data. To prevent this, we employ regularization techniques such as **Norm-Regularization**, **Dropout**, and **Early Stopping** to discourage overly complex models and improve generalization on unseen data.

Honor Code

I pledge my honor that this lecture summary is my own work and adheres to the guidelines in the instructions.
- Daniyal Shahid Iqbal